Reimagining Proof-of-Work with Elliptic Curve Discrete Logarithms

Ghazaleh Keshavarzkalhori, Biel Castellarnau, Jordi Herrera-Joancomartí, Cristina Pérez-Solà
Departament d'Enginyeria de la Informació i les Comunicacions
Universitat Autònoma de Barcelona (UAB)
Ghazaleh.Keshavarzkalhori@uab.cat, Biel.Castellarnau@uab.cat, Jordi.Herrera@uab.cat, Cristina.Perez@uab.cat

***Abstract*—Proof-of-work (PoW) forms the foundation of blockchain technology and plays a pivotal role in securing decentralized networks. Most existing PoW schemes rely on hash-based computations, which, while effective for achieving consensus, perform work that has no intrinsic utility beyond the protocol itself. In this paper, we first review the fundamental properties required of PoW mechanisms and then propose a new scheme based on the Elliptic Curve Discrete Logarithm Problem (ECDLP) that satisfies these criteria. We further analyze the computational complexity of our approach and compare its performance with conventional hash-based schemes. Our results show that it is feasible to design a secure PoW mechanism grounded in a well-studied mathematical problem, offering an alternative to purely hash-based work while preserving the properties essential to blockchain consensus.**

***Index Terms*—proof-of-work, ECDLP, blockchain**

I. Introduction

Proof-of-work (PoW) mechanisms enable a prover to present evidence of having performed a prescribed amount of computational effort. Asymmetry is at the core of proof-of-work design: verification of the performed work is highly efficient, whereas generating it requires substantially greater computational cost.

Although PoW is now predominantly associated with blockchain consensus protocols, the concept predates blockchain applications and was originally proposed in the context of mitigating abuse in distributed systems, such as spam prevention and access throttling [1].

Most existing PoW constructions employ cryptographic hash functions as their core computational task. These designs benefit from well-understood security properties and straightforward verification. However, hash-based schemes inherently require computations with no value beyond the proof itself. This characteristic has motivated research into PoW mechanisms that introduce additional structure or potential utility, while retaining the asymmetry required for practical deployment.

In this work, we examine the abstract properties that a generic PoW scheme must satisfy and explain how these properties manifest in the context of blockchain systems. Based on this analysis, we propose a new PoW construction derived from the Elliptic Curve Discrete Logarithm Problem (ECDLP). The scheme retains efficient verification and satisfies the properties necessary for blockchain mining.

The contributions of this paper are thus the enumeration of the requirements for PoW mechanisms in blockchain-specific contexts, the design of a scheme instantiated from ECDLP that meets these requirements, and the analysis of the limitations of this design.

The remainder of the paper is organized as follows. Section II reviews the necessary background, including the ECDLP and standard mechanisms for hashing to elliptic-curve points. Section III introduces proof-of-work and outlines the properties required of PoW schemes in blockchain settings. Section IV then presents our ECDLP-based PoW construction and Section V analyzes its computational complexity. Then, Section VI surveys related work and highlights the main differences relative to our proposal. Finally, Section VII offers concluding remarks and directions for future research.

II. Background

This section presents the essential background for building our proposal, focusing on the Elliptic Curve Discrete Logarithm Problem (ECDLP) and the RFC 9380 procedures for generating elliptic-curve points from arbitrary data.

A. The Elliptic Curve Discrete Logarithm Problem

Cyclic groups over elliptic curves may be defined by selecting a generator point, G, and taking all the powers of G under the elliptic-curve group law:

$$\mathbb{G} = \{P \text{ such that } P = G^x, \forall x \in \mathbb{Z}_q\}$$

where q is the order of the generator.

Solving the ECDLP in this context refers to the problem of finding the element x used to generate a given point $P \in \mathbb{G}$. While this multiplicative notation aligns more directly with the terminology of the discrete logarithm problem, we adopt additive notation for the remainder of this work.

Since ECDLP is known to be a computationally hard problem, many cryptographic schemes hide secret values as the value used to generate a public point to make it computationally infeasible for any entity to get access to the secret.

The **naive method** of breaking the ECDLP is to try all possible $x \in \mathbb{Z}_q$, computing $x \cdot G$, up until one coincides with P.

Shanks' algorithm [2] enhances the efficiency of this attempt by searching for points that would add up to P to which we know the field elements. Doing so reduces the search for x to only $\sqrt{q}$, making the search much more efficient. Although this effort reduces the search to only the square root of the size of the field, the storage needed for the algorithm is of order $O(\sqrt{q})$.

Pollard's rho-algorithm [3], on the other hand, removes the need for such large storage and guarantees the search for x with $\sqrt{q}$. This is done by choosing a random function that has the same domain and range. In each iteration, a field element

is transformed into another using this function. After each transformation, the algorithm checks whether the resulting element satisfies a predefined condition that qualifies it as a distinguished point (for example, having a fixed number of zero-leading bits). If the condition is met, the point, along with its associated data, is stored in a shared list accessible to all parallel processes. The storage needed is $O(d)$ where d is distinguished points in the list. A collision is detected when a newly computed distinguished point matches one already in the list. This collision can then be used to solve for the discrete logarithm.

Given G and $Q = k \cdot G$ points on the elliptic curve, where k is a positive integer and the discrete algorithm we want to find (which must be less than order q), the Pollard's rho-algorithm is defined as:

1) Each point R in the sequence is represented as: $R = aG + bQ$
2) If R is a distinguished point, we store the tuple (R, a, b) in the shared list.
3) We compute the next point using $(R, a, b) = nextPoint(G, Q, R, a, b)$ and start again until a distinguished point is found.
4) Let the elliptic curve group $E(\mathbb{Z}_p)$ be partitioned into three subsets: S_1, S_2, S_3. The $nextPoint$ function is defined as:

$$f(R_i) = \begin{cases} Q + R_i, & \text{if } R_i \in S_1 \\ 2R_i, & \text{if } R_i \in S_2 \\ G + R_i, & \text{if } R_i \in S_3 \end{cases}$$

5) When a collision is found, where $R = aG + bQ$ is the current point and $R^* = cG + dQ$ is a previous point saved on the shared list, we can solve for k as: $k = \frac{a-c}{d-b} \mod q$, provided $d \not\equiv b \pmod q$. This yields the discrete logarithm k such that $Q = kG$.

A variety of alternative methods have been developed to enhance the efficiency of solving the ECDLP in classical computation, as comprehensively discussed by Hankerson et al. [4]. These algorithms are still computationally heavy for processing practical secure curves.

Pollard's lambda method (also known as the ***kangaroo method***) is more efficient than Pollard's rho method in cases where the discrete logarithm x is known to lie within a restricted range, typically $x \in [0, b)$ with $b < q$. In practical implementations, such a constraint allows for faster computation. The method can be visualized as a number line of points $G, 2G, 3G, \ldots$. Imagine two kangaroos on this line: a *tame kangaroo* and a *wild kangaroo*. The goal is for the wild kangaroo to catch the tame one.

1) The *tame kangaroo* starts at a known point $R_0 = bG$ and performs a sequence of jumps. The next location of the kangaroo is determined by $R_{i+1} = R_i + f(R_i)P$ where the function $f(R_i)$ determines the jump distance.
2) The *wild kangaroo* starts at $R'_0 = Q$ and performs jumps using the same function $f(R)$. If the wild kangaroo's total distance exceeds that of the tame kangaroo, it restarts from a new point $R'_0 = Q + zG$, for a small known vale of z.

The process continues until the wild kangaroo lands on the same point as the tame kangaroo, at which point the discrete logarithm can be calculated from the difference of the distances traveled by the two kangaroos.

B. Hashing to Elliptic Curves (RFC 9380)

In order to ensure that a calculated point belongs to a curve without excessive computation, the mechanism described in *RFC 9380: Hashing to Elliptic Curves* [5] is used.

This RFC defines a `hash_to_curve(msg: bytes)` function that maps a message to a point P on a curve. The process begins with `hash_to_field(msg, count)`, where `count=` 2. It returns two field elements in a finite field $\mathbb{F}_p$. This is done using an `expand_message(msg,DST,l` function. `msg` is a byte string containing the message to hash, `DST` is a byte string that acts as a domain separation tag and `l` is the number of bytes to be generated. The function produces a uniformly random byte string using a cryptographic hash function H that outputs b bits.

Next, each field element is passed to `map_to_curve`(v), which returns a point Q on the curve. The RFC focuses on short Weierstrass curves, allowing the use of the Simplified Shallue–van de Woestijne (sSWU) [5] method. This requires curve parameters A, B with $AB \neq 0$, and a constant Z that must be a non-square in $\mathbb{F}_q$. Since two field elements were returned in the previous function, a point for each element will be created. These two points are added to get the final desired point R.

Finally, the `clear_cofactor(R)` function ensures the point lies in the prime-order subgroup G. In our case, since the curve has prime order n, the cofactor is $h = 1$, so $G = E(\mathbb{F}_q)$ and no clearing is needed.

The `hash_to_curve` algorithm is thus used to convert an arbitrary-length byte string `msg` into P, a point in a curve E:

1) Extract the field elements:

$$\mathbf{u} = \texttt{hash_to_field}(\texttt{msg}, 2)$$

2) Transform the field elements into points in the curve[1]:

$$R_0 = \texttt{map_to_curve_sSWU}(u[0]),$$
$$R_1 = \texttt{map_to_curve_sSWU}(u[1])$$

where sSWU is the Simplified Shallue–van de Woestijne mapping for $y^2 = x^3 + Ax + B$.

3) Add the points:

$$P = R_0 + R_1$$

4) Return the point of the curve P.

III. Proof-of-work for blockchain

Proof of Work (PoW) was designed to ensure that a certain amount of computational effort has been expended to complete a task that is easy to verify. This asymmetry between expensive computation and cheap verification is central to its security model, especially in blockchain systems like Bitcoin.

The formal description of a PoW scheme comprises three algorithms:

[1]The RFC generates two points and sums them, rather than producing a single point directly, to ensure that the resulting output is statistically close to a uniform distribution.

- `c = Gen(`d`)`: A deterministic algorithm with unpredictable output that generates a challenge c for a given difficulty d.
- `s = Solve(`c`)`: A computationally expensive algorithm that attempts to solve the challenge, producing a solution s.
- `TRUE/FALSE = Verify(`c, s`)`: A fast algorithm that checks whether s is a valid solution to c.

The key requirement is that `Gen` and `Verify` must be fast, while `Solve` must be computationally expensive. This ensures that the solution s shows that a certain amount of work was done. Moreover, `Verify` must be able to confirm that c was generated in a way that required this effort, preventing trivial or precomputed challenges.

When deployed in a blockchain environment, a proof-of-work mechanism is required to have the following properties:

- **Difficulty adjustment:** When additional computational power enters the network, the average time required to solve a challenge decreases; conversely, when mining power drops (because miners leave or hardware becomes unavailable) the average time increases. A reliable difficulty-adjustment mechanism is therefore required to keep the solving times of the challenge c near their target value, thereby ensuring stable block production despite fluctuations in total computational capacity. Moreover, the adjustment must be sufficiently granular to keep actual block times close to the theoretically defined target.
- **Non-reusability and input uniqueness:** The solution s must be intrinsically linked to a specific instance of the problem (e.g., a specific block header). This prevents reuse or precomputation of solutions, ensuring that each PoW is fresh and tied to the current state of the chain.
- **Linkability to the block:** The challenge c must include the previous block's hash and current transaction data, making each PoW cryptographically linked to the block it secures. This chaining prevents miners from predicting or preparing future blocks in advance.
- **ASIC resistance (desirable):** Ideally, PoW should be resistant to specialized hardware (ASICs), promoting decentralization by allowing general-purpose CPUs to compete fairly. This supports the "one-CPU-one-vote" [6] philosophy and reduces the risk of mining centralization.

Together, these properties make PoW a robust mechanism for securing decentralized systems, ensuring fairness, verifiability, and resistance to manipulation.

In Bitcoin, PoW involves finding a nonce such that the hash of the block header meets a difficulty target (e.g., has a certain number of leading zeros):[2]

$$hash(\text{ header } || \text{ nonce }) < \text{difficulty_target}$$

Generating such a hash is computationally expensive. However, once a valid hash is found, it can be efficiently and independently verified by any node, incurring minimal verification cost.

Although Bitcoin's proof-of-work effectively demonstrates that significant computational effort has been expended and allows for rapid verification, the work itself consists solely of performing hashing operations, which have no function beyond satisfying the requirements of the proof-of-work protocol.

[2]Technically, the nonce is included in the header, but we have opted for this formulation to simplify the explanation.

IV. ECDLP-BASED PROOFS-OF-WORK

The overall goal of the PoW algorithm presented in this paper is to generate ECDLP instances whose difficulty can be adjusted while remaining cryptographically bound to the specific block being mined (see Section III). Adjustable difficulty is achieved by varying the underlying cyclic group in which the ECDLP is defined, accomplished through the careful selection of the elliptic curve on which the problem is instantiated. The binding to a particular block is ensured by deriving the target point for the discrete logarithm computation directly from the current block header.

In general, PoW blockchains define epochs (of length Δ_{Epoch} blocks) during which the difficulty d of the PoW is constant. At the end of each epoch, difficulty is adjusted to account for computational power changes in the network (see Fig 1).

The rest of this section covers how blocks are generated, solved, and verified. Phase 0 first elaborates on how the difficulty is set at the start of each epoch, while phases 1-3 dig deeper into how a challenge is set for the miner to solve, how it solves it, and how the rest of the miners can verify the block upon receiving it.

Phase 0: Difficulty Setting

This phase is only carried out at the start of a new epoch. The goal of this phase is to select a, b and p defining an elliptic curve over which the ECDLP is going to be solved:

$$E : y^2 = x^3 + ax + b \mod p$$

In our protocol, selected curves are always of prime order. This guarantees that all points on the curve lie in a single cyclic group, and that any non-identity point can serve as a generator. Consequently, every ECDLP instance is bounded by the order of the curve, which is chosen to reflect the desired difficulty.

In this phase, we employ the algorithm proposed by Bröker et al. [7] to generate a Hilbert class elliptic curve [3] containing exactly q points, where q is prime number whose size is defined by the current difficulty:

1) Let `data` be the previous block's header.
2) $h = hash(\texttt{data})$
3) $n = 2^{2d} + (h \mod 2^{2d})$
4) $q = ComputeNextPrime(n)$
5) $(a, b, p) = ConstructEllipticCurve(q)$

The protocol then selects a point on the curve to be used as the generator for the ECDLP instance. This is accomplished via the `hash_to_curve` algorithm (Section II-B), using the block header of the last block from the previous epoch as the `msg` and `ECDLP_NEW_EPOCH` as the domain-separation tag.

Once the elliptic curve and generator for the epoch are defined, miners derive the challenge for the current block.

[3]To the best of our knowledge, no algorithms more efficient than the classical approaches—such as Shanks's method or Pollard's rho algorithm—are currently known for solving the ECDLP on these curves, thereby supporting their suitability and security for our intended application.

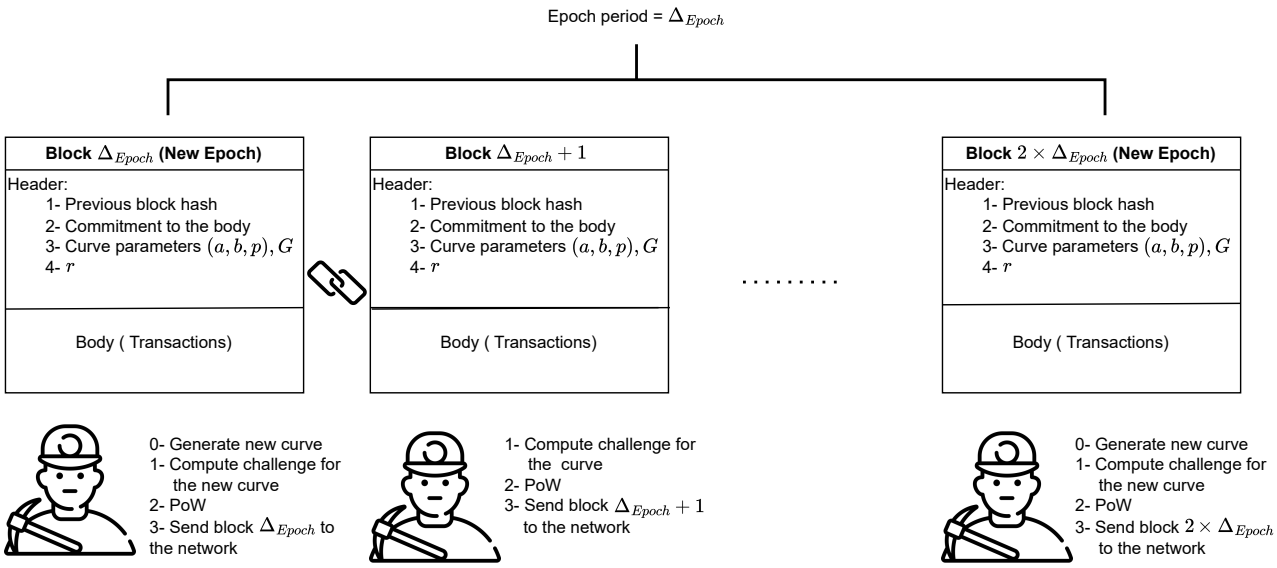

Fig. 1. The structure and types of blocks existing in the blockchain.

Phase 1: Challenge Computation

In the proposed PoW, the challenge for each block is computing the discrete logarithm of a point P in the current epoch's elliptic curve. The point P is deterministically derived from the block to ensure likability and non-re-usability (as defined in Section II-A).

Specifically, the point P is derived from calling the `hash_to_curve` algorithm (Section II-B), using the current block header as the `msg` and `ECDLP_NEW_BLOCK` as domain separation tag:

1) Let `msg` be the current block's header, with the field r set to 0.
2) $P = hash_to_curve(\texttt{msg}, \texttt{ECDLP_NEW_BLOCK})$

Phase 2: Proof of Work

In this phase, the miner solves the discrete logarithm of the point P over the epoch's curve, that is, determines the integer r such that $P = r \cdot G$ over the curve E.

To do so, he may use one of the algorithms for solving the ECDLP presented in Section II-A.

The first miner to solve the challenge will then update the header of the block to include the solution of the challenge r and broadcast its block to the network.

Phase 3: Verification

Each miner receiving a new block must validate if the transactions are valid as well as check if the challenge of the block has been solved and created correctly. Miner M' receiving the block from M will carry out the following checks:

1) Check that the curve over which the ECDLP is solved, is correctly derived from the difficulty adjustment algorithm.
2) Check that the challenge P is correctly derived from the current block header. To recompute P, the validators set $r = 0$ in the header when calling the `hash_to_curve` function.
3) Check that the original r is a valid solution of the ECDLP instance, that is, $P = r \cdot G$.

If any of the checks fail, the validation process is terminated and the block is rejected. If all checks succeed, the block is accepted into the network, and mining proceeds on top of it.

V. Complexity analysis

We now analyze the computational complexity of the proposed PoW algorithm and its dependence on the difficulty parameter. Let E be the elliptic curve selected for the current epoch, with $q = |E(\mathbb{F}_p)|$ denoting the number of group elements; the ECDLP is solved over this group. Under the protocol definition, q is chosen as the smallest prime greater than $2^{2d} + (h \mod 2^{2d})$ where h denotes the hash of the last block in an epoch.

Solving the ECDLP for a point $P \in E$ incurs a time complexity of $O(\sqrt{q})$. Under the simplifying assumption that $q \approx 2^{2d}$, block generation therefore requires $O(2^d)$ time and uses constant space $O(1)$.

For comparison, in Bitcoin the computational complexity of the proof-of-work mechanism is given by $O(2^d)$ where d denotes the network difficulty parameter. The dominant operation consists of repeated evaluations of the double SHA-256 hash function, as opposed to elliptic curve scalar multiplications used in the proposed algorithm.

It is important to note that the computational complexity of auxiliary algorithms employed during block generation, such as `HashtoCurve` and `ConstructEllipticCurve` is negligible compared to the PoW. These algorithms exhibit complexities on the order of $O((\log q)^\alpha)$ where $\alpha \in \{1, \ldots, 4\}$, which is insignificant relative to the exponential cost of PoW.

VI. Related work

Several mechanisms have been proposed in the literature to demonstrate the consumption of different types of computational resources, including proof of memory [21], proof of storage [22], and proof of bandwidth [23].

Mechanism	Family	Problem	Example
Classic hashing [1]	Hash-based	Find a hash below a target value	D: Bitcoin (SHA256) [6]
Memory-hard hashing	Hash-based	Find a hash below a target value	D: Litecoin (Scrypt) [8], [9]
Equihash [10]	Hash-based	Solve generalized birthday problem	D: Zcash [11]
Prime chains	Number-theoretic	Find chains of prime numbers	D: Primecoin (Cunningham/Bi-twin chains) [12], Riecoin (prime constellations) [13]
Integer factorization	Number-theoretic	Factor a composite number	D: Factorn [14], [15]
Discrete logarithm	Number-theoretic	Solve a discrete logarithm	T: Meneghetti et al. [16]
Repeated squaring	Iterative function-based	Sequential modular arithmetic	D: Chia [17]
Cuckoo Cycle [18]	Graph-based	Find a cycle of fixed length in a bipartite graph	D: Grin [19]
Dominating set	Graph-based	Find valid dominating set	T: ScaloWork [20]

TABLE I
EXISTING PROOF-OF-WORK MECHANISMS. IN THE EXAMPLE COLUMN, D STANDS FOR A DEPLOYED PROJECT, WHEREAS T STANDS FOR A THEORETICAL PROPOSAL (NOT YET DEPLOYED IN AN EXISTING BLOCKCHAIN PROJECT).

In this work, we focus exclusively on proof-of-work (PoW), which requires a participant to perform a computationally intensive task that can be efficiently verified. Existing PoW constructions can be broadly categorized according to the nature of the underlying computational problem (see Table I for a summary). Hash-based schemes remain the most widely deployed due to their simplicity and well-understood security properties. Other approaches include number-theoretic schemes, which rely on arithmetic problems such as finding large primes; iterative function–based schemes, which require repeated application of a function with certain cryptographic properties; and graph-based schemes, which use hard problems defined over graph structures. Our work builds on this landscape by proposing a PoW mechanism based on the elliptic curve discrete logarithm problem.

A. DLP-based proof of work

Although relatively few, some prior works have explored the design of proof-of-work schemes based on the discrete logarithm problem.

Hastings et al. [24] propose a PoW algorithm to motivate research be done towards advancing the computation of DLPs. In their work, they take advantage of Pollard's rho algorithm to find distinguished points with a specific number of starting zeros in their bit representations. Hastings suggests to set the starting point of the Pollard's algorithm to the hash of the previous block and a nonce to avoid reusability and provide freshness for the miners as well as setting an upper bound for the number of steps carried out for each hash and nonce. This will assure that the validation of the PoW will be fast and thorough.

The ECDLP-Based PoW Model [16] proposes having two types of blocks. Epoch blocks select elliptic-curve parameters. In addition to the header and transactions, the miner computes a prime p, an elliptic curve E, and a base point $G \in E(\mathbb{F}_p)$. As part of the proof-of-work, the miner must provide integers N_1 and N_2 such that $N_i G$ are valid points on E. These blocks appear every 2016 blocks, mirroring the difficulty-adjustment schedule of Bitcoin. Standard blocks contain only the usual header and transactions and inherit p, E, and G from the last epoch block.

The validation conditions are based on a deterministic function $\mathsf{P_Gen}$ mapping a hash to a point on E. A block is valid only if two integers (N_1, N_2) satisfy

$$\mathsf{P_Gen}(H(h_{prev}), E) = N_1 G,$$
$$\mathsf{P_Gen}(H(M), E) = N_2 G.$$

where h_{prev} is the previous header and M the Merkle root.

The parameter p is generated so that the resulting curve depends on the current difficulty d. Starting from the hash of the previous header h, a candidate prime is computed as $p = h \bmod 2^{2d}$; then the function $\mathsf{Next_Prime}$ is applied to obtain the next smallest prime at least this large. Primes with undesirable structure (e.g., Crandall, generalized Mersenne, or Montgomery-friendly forms) are rejected, and the process is repeated with re-hashing until an acceptable p is found.

Given p and h, the curve coefficients are defined by $A_E = H(h + i)$ and $B_E = H(A_E)$ for an integer counter i, which produce the curve

$$E : y^2 = x^3 + A_E x + B_E \pmod p.$$

The curve is accepted only if it meets the standard security properties, including a prime group order distinct from p, embedding degree exceeding 20, and a complex multiplication discriminant larger than 2^{40}. If any condition fails, i increases and the coefficients are recomputed.

The generator point is finally obtained as

$$G = \mathsf{P_Gen}(H(p \parallel A_E \parallel B_E),\, E)\,.$$

B. Comparison with existing protocols

Previous works have explored alternatives to hash-based designs by using elliptic curve structures, with the objective of making the computational effort more meaningful. However, these protocols are significantly different in how they define mining tasks, manage curve generation, and adjust difficulty.

The proposed approach in [4] does not require miners to solve the Elliptic Curve Discrete Logarithm Problem (ECDLP); instead, the objective is limited to identifying distinguishable points on the curve. To achieve this, the system targets large elliptic curves while adjusting the difficulty so that the challenge remains computationally feasible—miners

only need to locate unique points for each block. This design is appealing because, if applied at scale, it would enable the network to collectively work on curves of near-practical size. However, the authors assume that a curve will eventually be solved after sufficient cumulative effort, yet they do not specify a mechanism for transitioning to a new curve once the current one becomes unusable, leaving a gap in the protocol's adaptability.

In [16] curve generation is performed by deterministically constructing a secure elliptic curve over a prime field $\mathbb{F}_p$, where the prime p is computed as the next prime greater than $h \mod 2^{2d}$, h being the hash of the previous block. Although this process is deterministic, the resulting prime may be relatively small, and the procedure iterates until a curve satisfying security requirements is obtained—without any explicit upper bound on the number of iterations. On the other hand, each miner receiving an epoch block is required to validate both the primality of p and the security properties of the curve, which introduces significant computational overhead.

Furthermore, while the design assumes that network difficulty is tied to operations on elliptic curve points, it incorrectly associates this difficulty with the size of the prime field rather than the actual group order of the curve. According to Hasse's theorem, the group order lies within the interval $(p+1-2\sqrt{p},\, p+1+2\sqrt{p})$, meaning that the field size and group size are not equivalent hence difficulty is not the same as the scheme is setting it to be.

VII. Conclusions and future work

In this paper, we have introduced an ECDLP-based proof-of-work mechanism as an alternative to traditional hash-based schemes. Using elliptic curve operations and the RFC9380 deterministic hash-to-curve mapping, the proposed design ensures that the blocks are linked and maintain the asymmetry required for a PoW (a challenge hard to solve and a verification easy to execute).

By introducing an elliptic curve–based challenge, the computational effort serves a dual purpose: not only to secure the network, but also to generate new elliptic curves that could have value in other domains[4]. Unlike traditional hash-based schemes, which produce results with no utility beyond consensus, elliptic curve challenges create mathematical artifacts that could help evaluate the hardness of elliptic-curve groups (or the security margins of specific parameters) and the performance of algorithms.

With respect to the design, future work will first focus on evaluating the granularity provided by our scheme to determine whether it is sufficient for blockchain applications, particularly in maintaining stable block production under varying network conditions. Additionally, subsequent research will involve implementing and testing a prototype of the protocol under real-world conditions to assess its practical performance, efficiency, and robustness.

Acknowledgements

This work is funded by the projects PID2021-125962OB-C33 SECURING/NET and PID2024-156914OB-C43 SAFE-BLOCKCHAIN, funded by the Ministerio de Ciencia e Innovación, la Agencia Estatal de Investigación and the European Regional Development Fund (ERDF); the DANGER Strategic Project of Cybersecurity C062/23, funded by the Spanish National Institute of Cybersecurity through the European Union - NextGenerationEU and the Recovery, Transformation and Resilience Plan; and the Catalan Agència de Gestió d'Ajuts Universitaris i de Recerca (AGAUR) grant SGR2021-00643.

[4]For example, this effort could be applied to the exploration of prime gaps, such as investigating intervals that might violate Bertrand's postulate while selecting prime parameters for each difficulty adjustment.

References

[1] A. Back *et al.*, "Hashcash-a denial of service counter-measure," 2002.
[2] D. Shanks, "Class number, a theory of factorization, and genera," in *Proc. Symp. Math. Soc., 1971*, vol. 20, 1971, pp. 415–440.
[3] J. M. Pollard, "Monte carlo methods for index computation," *Mathematics of computation*, vol. 32, no. 143, pp. 918–924, 1978.
[4] D. Hankerson, S. Vanstone, and A. Menezes, *Guide to elliptic curve cryptography*. Springer, 2004.
[5] A. Faz-Hernandez, S. Scott, N. Sullivan, R. Wahby, and C. Wood, *RFC 9380 Hashing to Elliptic Curves*. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9380.pdf
[6] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," *Available at SSRN 3440802*, 2008.
[7] R. Broker and P. Stevenhagen, "Constructing elliptic curves of prime order," *arXiv preprint arXiv:0712.2022*, 2007.
[8] J. Choe, T. Moreshet, R. I. Bahar, and M. Herlihy, "Attacking memory-hard scrypt with near-data-processing," in *Proceedings of the International Symposium on Memory Systems*, 2019, pp. 33–37.
[9] L. Foundation, "Litecoin developer guide: Proof of work," https://litecoin.info/docs/key-concepts/proof-of-work, 2024.
[10] A. Biryukov and D. Khovratovich, "Equihash: Asymmetric proof-of-work based on the generalized birthday problem," *Ledger*, vol. 2, pp. 1–30, 2017.
[11] D. Hopwood, S. Bowe, T. Hornby, N. Wilcox *et al.*, "Zcash protocol specification," *GitHub: San Francisco, CA, USA*, vol. 4, no. 220, p. 32, 2016.
[12] S. King, "Primecoin: Cryptocurrency with prime number proof-of-work," *July 7th*, vol. 1, no. 6, 2013.
[13] T. R. Developers, "The stella pow," https://riecoin.xyz/Guides/Stella/, 2025.
[14] F. development team, "Factorn docs," https://docs.fact0rn.io/, 2024.
[15] E. Liones, "FACT0RN: Integer Factorization as Proof of Work," Tech. Rep., 2022.
[16] A. Meneghetti, M. Sala, and D. Taufer, "A new ecdlp-based pow model," *Mathematics*, vol. 8, no. 8, p. 1344, 2020.
[17] C. N. Inc, "Chia documentation," https://docs.chia.net/chia-blockchain/consensus/proof-of-time/, 2025.
[18] J. Tromp, "Cuckoo cycle: A memory bound graph-theoretic proof-of-work," in *International Conference on Financial Cryptography and Data Security*. Springer, 2015, pp. 49–62.
[19] G. development team, "Grin documentation," https://docs.grin.mw/about-grin/proof-of-work/, 2024.
[20] D. Chatterjee, A. Majumder, and S. Mazumdar, "Scalowork: Useful proof-of-work with distributed pool mining," *arXiv preprint arXiv:2504.14328*, 2025.
[21] M. Abadi, M. Burrows, M. Manasse, and T. Wobber, "Moderately hard, memory-bound functions," *ACM Transactions on Internet Technology (TOIT)*, vol. 5, no. 2, pp. 299–327, 2005.
[22] G. Ateniese, R. Burns, R. Curtmola, J. Herring, L. Kissner, Z. Peterson, and D. Song, "Provable data possession at untrusted stores," in *Proceedings of the 14th ACM conference on Computer and communications security*, 2007, pp. 598–609.
[23] B. Chrobot, D. Lax, and D. O'Donovan, "Torcoin," MIT 6.858: Computer Systems Security, Fall 2014, 2014. [Online]. Available: https://css.csail.mit.edu/6.858/2014/projects/bchrobot-ynnad19-dereklax.pdf
[24] M. Hastings, N. Heninger, and E. Wustrow, "Short paper: The proof is in the pudding: Proofs of work for solving discrete logarithms," in *International Conference on Financial Cryptography and Data Security*. Springer, 2019, pp. 396–404.